Applying Meta-Evaluation Benchmark Reviews to Requirement Decisions: An Executable Method

Anonymous Author(s)

Affiliation Withheld for Double-Blind Review
contact@example.org

Abstract. Context and motivation: As AI coding agents accelerate software patching, the engineering bottleneck shifts upstream to validating whether a candidate agent satisfies project requirements. Teams often use public leaderboards (SWE-bench Verified) or automated audits as acceptance tests, yet scores can be distorted by overbroad tests, exploitable tests, unisolated runners, and pretraining exposure.
Question/problem: Benchmark audits flag flawed tasks and checklists score documentation, but neither links review evidence to a requirement threshold, leaving open how reviewing a benchmark with MESA or ABC changes a decision beyond an audit (RQ1) or agrees with post-cutoff testing (RQ2).
Principal ideas/results: We implement a three-level decision engine checking input integrity, evidence status, and Wilson bounds while returning every blocking reason and removing defective tasks. Applied to a SymPy requirement ($\geq$ 70% resolved in 15 minutes at 95% confidence; 2 agents, 20 SWE-bench Verified source tasks, 20 post-cutoff target tasks), executed checks found cheating patches passing hidden tests on 17/20 source tasks (6 breaking issue-stated behaviors on signed B07, plus 1 B05 task) and runner hijacks succeeding in 9/9 attempts (B28a). With all nine forms and the requirement signed, strict and lenient MESA policies return **Inadmissible** (**No decision**), while removing 6 tasks leaves $N = 14$ for audit-only filtering (Accept $14/14$; Reject $\leq 6/14$) versus $N = 20$ for raw and ABC (Accept $\geq 19/20$; Reject $\leq 9/20$; 97.2% Inconclusive at $p = 0.75$).
Contribution: (1) Executable schemas and decision engine; (2) executed B05, B07, and B28a checks and signed SymPy review records; and (3) a pre-registered protocol across five method families (six outputs), with sessions pending freeze.

Keywords: Requirements Engineering for AI · AI Benchmarks · Acceptance Testing · Evidence Reuse · Traceability · Coding Agents.

1 Introduction

As autonomous AI coding agents accelerate patch generation and bug fixing, the software engineering bottleneck shifts upstream to validating whether a candidate agent satisfies system capability requirements [9,14]. Because running direct acceptance tests across a project's workload is costly, teams frequently substitute public benchmark scores—such as leaderboard resolve rates on SWE-bench Verified—as fast acceptance evidence [7,11,25]. Yet compressing requirements validation into a benchmark resolve rate exposes teams to three evidential risks: task tests may reject valid fixes or accept

incomplete patches [5,20], grading harnesses may run untrusted patches without isolating the test runner [29], and candidate models may have memorized public benchmark issues and fixes during pretraining [21].

We anchor our method and evaluation on a concrete operational requirement for the SymPy symbolic mathematics repository: *"Working alone on a bug report from the SymPy repository, the coding agent changes the code so that the tests the maintainers later wrote for that bug pass, for at least 70% of bug reports, within 15 minutes each."* To see how benchmark evidence interacts with this requirement, consider two SymPy issues from SWE-bench Verified:

- **Task illustration (`sympy__sympy-21930` and `sympy__sympy-18199`):** In `sympy__sympy-21930` (used in our procedure rehearsal outside the study sample), the issue reports that LaTeX printing of the square of `Bd(a)` produces an invalid double superscript; the agent receives the issue text and pre-fix repository and must produce a patch that passes the maintainers' hidden regression tests. In `sympy__sympy-18199` (one of the 20 study source tasks), the issue asks for $x \equiv 0 \pmod{p}$ root checking in `nthroot_mod`, but the hidden test suite also requires composite-modulus handling never requested in the issue text, rejecting a minimal fix of the stated bug (a **B05** defect) while simultaneously failing to check another case stated in the issue (a **B07** defect).
- **Source tasks vs. target tasks:** *Source tasks* are 20 SymPy tasks sampled from SWE-bench Verified (issues from 2016–2023) that represent the public benchmark evidence a practitioner would inspect. *Target tasks* are 20 fresh SymPy bug fixes merged between 13 June and 5 November 2026—after the stated training cutoffs of both evaluated models—which form our *direct test* of the requirement.

Throughout this paper, seven terms have fixed definitions: (1) *Coding agent* (recorded as a *configuration*): an AI model paired with its command-line tool and execution environment. (2) *Issue* and *task*: an issue is a bug report on SymPy's tracker; a task is an issue prepared with pre-fix repository code and hidden maintainer regression tests (`FAIL_TO_PASS` and `PASS_TO_PASS`). (3) *Session*: one isolated execution of a coding agent on one task. (4) *Resolved*: the session's patch passes all tests in the grading harness within the 15-minute (900-second) limit and satisfies session validity rules (Section 4.4). (5) *Rating form*: the record of one MESA criterion's evidence *level*, check *outcome*, check kind, scope, detection limits, and task defects (Section 2.1). (6) *Evidence status* (Tier 3 qualification gate: **Admissible**, **Inquiry Required**, or **Inadmissible**): gates whether benchmark evidence can support a requirement decision. (7) *Score decision* (Tier 3 requirement acceptance result: **Accept**, **Reject**, **Inconclusive**): computed from the 95% Wilson score interval on retained tasks when evidence is **Admissible**; otherwise the engine returns **No decision** (withholding judgment due to evidence defects).

MESA, presented in a companion submission, unifies seven benchmark review frameworks into 30 shared concerns and a 39-criterion rubric (8 Part A descriptive facets and 31 Part B evaluative criteria) and proposes a conceptual Requirements Traceability Matrix. This paper turns that conceptual design into an executable decision method and evaluates its outputs on SWE-bench Verified (SymPy), addressing two gaps in existing assurance and benchmark evaluation literature:

Table 1. Artifact boundary and study status prior to protocol freeze.

Category	Scope and Current State
Adopted from MESA	Rubric criteria, six evidence levels, rating record fields (outcome, check kind, scope, detection limits, task defects), and three-stage admissibility and Wilson score structure.
Implemented & tested in this paper	Versioned case, requirement, and form schemas; three-level decision engine; strict/lenient policies; session ledger and scanner; replay builder; exact binomial power calculations (N = 14, 15, 18, 20, 30–120) and simulation; 181 automated unit tests.
Executed checks & signed records (done 9 Nov 2026)	Eligibility verification of all 75 source and 20 post-cutoff target SymPy tasks; published container checks; sandbox checks (v1–v8) and rehearsal on `sympy__sympy-21930`; 20 **B07** exploit attempts (17 passes; 6 issue-scope task defects); **B05** check on `sympy__sympy-18199`; 9 **B28a** runner checks; ABA overlap comparison; rating runs A and B, ABC run; signed forms, ABC, and requirement.
Pending after protocol freeze	Generation of frozen manifest; phase authorizations; execution of at most 90 coding-agent sessions (2 pilot, 40 source, 40 target, up to 8 reserve slots); and replay execution on collected source records.

Gap 1. Automated benchmark audits such as BenchGuard [28], ABA [30], and BenchJack [29] detect broken tests and harness flaws, while UTBoost [32] and SWE-bench+ [1] recompute resolve rates after filtering tasks, and ABC [34] scores checklist items as caveats. Among existing benchmark audits and assurance methods, none provides an executable gate that combines task-defect removal with criterion-level admissibility and confidence-bound thresholding for a system requirement. This motivates RQ1.

Gap 2. Studies of SWE-bench Live [33], SWE-rebench [3], and the SWE-Bench Illusion [15] show that leaderboard scores drop on post-cutoff tasks. Among existing methods, none has evaluated how requirement acceptance decisions derived from raw, audited, or reviewed benchmark tasks compare against direct acceptance testing of the same requirement on post-cutoff tasks from the target repository. This motivates RQ2.

1.1 Research Questions

- **RQ1 (Audit vs. Review):** How does reviewing a benchmark with MESA or ABC, beyond an automated audit, change the decision on whether a coding agent meets a requirement?
- **RQ2 (Evidential Transfer):** Does a decision from a reviewed benchmark agree with a direct test of the same requirement on post-cutoff repository tasks?

1.2 Artifact Boundary and Study Status

Table 1 separates what this paper adopts from MESA, what has been implemented, executed, and signed as of 9 November 2026, and what remains pending execution after the protocol freeze.

2 Background and Related Work

2.1 Foundations from the Meta-Evaluation Standard

The MESA rubric comprises eight Part A descriptive facets and 31 Part B evaluation criteria organized across Kane's four validity inferences: scoring, generalization, extrapolation, and decision [12]. A rating form evaluates one criterion on six levels (*Claimed*,

Documented, *Verified*, *Contradicted*, *Undocumented*, and *Not applicable*) and records the check outcome (*passed*, *failed*, *not performed*, or *not recorded*), check kind (execution source: *reported*, *run*, *both*, or *none*), scope, detection limits (disclosing known check blind spots), task defects, and signers.

This paper extends MESA's conceptual traceability model with three operational rules for coding-agent evaluation: (1) **Signed transfer justification (benchmark relevance):** A source benchmark decision transfers to a target requirement only when accompanied by a signed transfer justification (explaining why past benchmark fixes apply to the target repository) whose source and target scopes match the case observations, repository, and configuration. (2) **Execution provenance per observation:** Every task observation carries a session identifier, log path, and SHA-256 log digest verified before scoring. (3) **Dual policy evaluation:** Each case is evaluated under a pinned *strict* and a pinned *lenient* policy (Section 3.3).

2.2 Related Work

Existing literature on AI evaluation and assurance spans three main areas: (1) *Machine learning assurance and formal specifications:* AMLAS [10], assurance contracts [27], safety case impact analyses [13], and transfer assurance [22] structure arguments for component reuse or retraining, but do not qualify external coding benchmarks. Pipeline evidence reuse [18] applies fail-closed release gates to data pipelines, VNVSpec [2] links requirements to verification runs with inconclusive results, and coding agent envelopes [8] solve runtime change obligations rather than qualifying design-time benchmark evidence. (2) *Guidance, quality models, and adoption frameworks:* NIST AI 800-2 [19] outlines language model evaluation targets without decision rules, general software engineering surveys [11] disclose benchmark gaps as limitations, DAMSA–SMS [26] applies Pareto screening without evidence withholding, and adoption studies [23] adapt GQM for organizational risks rather than requirements acceptance. (3) *Benchmark rubrics and automated audits:* On the benchmark side, ECBD [16] uses validity warrants to guide benchmark authoring, BetterBench [24] scores 46 benchmark quality criteria, Evaluation Cards [7] report metadata, and ABC [34] scores agentic checklists as caveats without decision thresholds. Automated benchmark audits—BenchGuard [28], ABA [30], and BenchJack [29]—flag broken tests (**B05**), test gaming (**B07**), and unisolated runners (**B28a**, **B28b**), alongside oracle analyses [5,20] and transcript scanners [17].

As shown in Table 2, however, automated audits do not evaluate benchmark uncertainty reporting (**B12**), benchmark exposure safeguards (**B14a**), or candidate-model pretraining contamination (**B14b**).

3 Method

3.1 Case and Rating Form Records

A *case* binds one requirement, one coding agent configuration, and the observations and review records used to judge it. Each case record carries a pinned schema version and six structured sections: (1) **Requirement:** Identifier, target repository, declared task

set, configuration, threshold θ, confidence level $1 - \alpha$, and minimum observation count $n_{\min}$ (checked at input integrity); (2) **Evidence Conditions:** Per-criterion sufficiency rules specifying the minimum evidence level, required check outcome, accepted scopes, required fields, and whether execution provenance is mandatory (checked at input integrity and evidence status); (3) **Configuration and Tasks:** Configuration identifier and declared task identifiers (checked at input integrity); (4) **Observations:** Task execution results, patch SHA-256 digests, and execution provenance linking each session to its log digest (checked at input integrity and evidence status); (5) **Rating Forms:** Criterion definitions, evidence levels, check outcomes, check kinds, scopes, procedures, detection limits, task defects, and signers (checked at input integrity and evidence status); and (6) **Transfer Justification:** Author-signed record stating why source benchmark tasks support the requirement on the target repository (checked at requirement decision).

3.2 Three Decision Levels

The decision engine evaluates a case in three sequential levels and reports all reasons at each level:

Level 1: Input integrity (*valid* or *rejected*). Before inspecting capability or review levels, the engine rejects a case under three categories of structural failure: (1) *mismatched identity or missing observations*, where an observation belongs to another configuration, repository, or undeclared task, or a declared task lacks an observation; (2) *corrupted or duplicated artifacts*, where a patch, artifact, or session log no longer matches its recorded SHA-256 file fingerprint (hash), or two observations reference the same patch; and (3) *invalid review or policy records*, where a rating form cites an unfrozen definition, a criterion has duplicate forms, or the policy digest differs from the pinned study policy. A rejected case receives no evidence status and no score decision.

Level 2: Evidence status (**Admissible**, **Inquiry Required**, **Inadmissible**, or *not applicable*). Because the rating *level* records what kind of check or documentation exists while the check *outcome* records whether that check passed or failed, a rating at level *Verified* whose check outcome is *failed* (such as our **B28a** grading runner check) makes the criterion **Inadmissible**. Specifically, for each criterion c in the requirement profile, the criterion status $v(c, r)$ is: (1) *not applicable* if the form marks c as *Not applicable* with a valid reason (forbidden for **B05**, **B12**, **B14a**, and **B14b**, which always apply when a configuration is named); (2) **Inadmissible** if the level is *Contradicted* or the check outcome is *failed*, the defect cites a verified location, and any counterexample patch was executed; (3) **Admissible** if the level meets the policy minimum, the check outcome is *passed*, the scope covers the configuration and all retained tasks, the required fields (procedure, check kind, location, detection limits) are present, and the author and model roles are recorded; and (4) **Inquiry Required** otherwise (including missing forms, unlocated defects, or unperformed checks). Across the profile, the aggregate evidence status is **Inadmissible** if any criterion is **Inadmissible**; else **Inquiry Required** if any criterion is **Inquiry Required**, required execution provenance is missing, or fewer than $n_{\min}$ tasks remain after removing task defects; else **Admissible**.

Level 3: Score decision (**Accept**, **Reject**, **Inconclusive**, or **No decision**). Tasks listed as defective on a signed **B05** or **B07** form with an executed counterexample and

Table 2. Selected meta-evaluation criteria, inferential stages, automated audit coverage, and policy qualification rules.

Id	Stage	Evaluation Question	Automated Audit Fit	Policy Qualification
B05	I_{extrap}	Do tests reject valid solutions?	BenchGuard / ABA ground truth	Strict & Lenient (*Verified*, passed)
B07	I_{score}	Does scoring accept wrong solutions?	BenchJack logic gaps / injection	Strict (*Verified*, passed); Lenient (*Documented*)
B12	I_{gen}	Does benchmark report uncertainty?	No automated audit coverage	Strict (*Verified*, passed); Lenient (omitted)
B14a	I_{extrap}	Does benchmark guard against exposure?	No automated audit coverage	Strict (*Verified*, passed); Lenient (omitted)
B14b	I_{extrap}	Could candidate model have seen tasks?	No automated audit coverage	Strict (*Verified*, passed); Lenient (omitted)
B28a	I_{dec}	Is grading environment isolated from code?	BenchJack isolation / untrusted output	Strict & Lenient (*Verified*, passed)
B28b	I_{dec}	Is shipped agent environment isolated?	BenchJack excessive permissions	Strict & Lenient (*Verified*, passed)

cited location are removed before any model score is inspected. For s resolved tasks out of n retained observations, the engine computes the 95% Wilson score interval [31,4]:

$$\text{LB, UB} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad \hat{p} = s/n,\ z = 1.95996. \tag{1}$$

The pre-gate score decision D_{perf} (the statistical score decision on clean tasks before checking evidence admissibility) is **Accept** if LB $\geq \theta$, **Reject** if UB $< \theta$, and **Inconclusive** otherwise. The source decision D_{src} and target requirement decision D_{req} are then:

$$D_{\text{src}} = \begin{cases} D_{\text{perf}} & \text{if evidence status is } \textbf{Admissible}, \\ \textbf{No decision} & \text{otherwise}, \end{cases} \qquad D_{\text{req}} = \begin{cases} D_{\text{src}} & \text{if transfer is justified}, \\ \textbf{No decision} & \text{otherwise}. \end{cases} \tag{2}$$

Here, the engine verifies structurally that a signed transfer justification exists and that its source and target scopes match the observations, repository, and configuration. When evidence is **Inadmissible** or **Inquiry Required**, the engine returns **No decision** rather than **Reject**, because flawed tests or missing checks do not prove that the candidate agent fails the requirement.

3.3 Selected Criteria Profile and Study Policies

We evaluate a restricted profile of seven MESA criteria spanning Kane's four inferential stages (Table 2): test errors that reject valid fixes (**B05**), scoring exploits that accept wrong fixes (**B07**), benchmark score uncertainty (**B12**), benchmark exposure safeguards (**B14a**), candidate-model pretraining exposure (**B14b**), grading harness isolation (**B28a**), and shipped agent environment isolation (**B28b**). Construct definition criteria (**B01**–**B04**) and domain transfer criteria (**B16**, **B23**) are omitted from the automated profile because the requirement names SymPy bug fixing directly and transfer is addressed through the signed transfer justification and tested empirically in RQ2.

Table 3. The five source decision method families (six evaluated outputs) compared in RQ1 and RQ2.

Output Code	Method Family	Tasks (N)	Cut Points ($\theta = 0.70$)	Task Filtering, Evidence Gate, and Known SymPy Status
`M2`	Raw resolve rate	20	Accept $\geq$ 19; Reject $\leq$ 9	Keeps all 20 source tasks; no task filtering and no evidence gate. Always issues a score decision.
`audit_only`	Executed B05/B07 task audit	14	Accept 14; Reject $\leq$ 6	Removes the 6 tasks with located B05/B07 test defects; applies no evidence gate. Equals pre-gate score decision (`M3_withheld`, counterfactual clean-task decision without evidence gating).
`ABA`	Published ABA audit [30]	18	Accept $\geq$ 17; Reject $\leq$ 8	Removes the 2 source tasks flagged with a major finding in ABA (`sympy__sympy-13031`, `-18199`); applies no evidence gate.
`M3_strict`	MESA (strict policy)	14	Withheld (**No decision**)	Removes the 6 B05/B07 defective tasks and checks all 7 criteria. Status: **Inadmissible** (**B14a** and **B28a** failed; **B12** and **B14b** inquiry required).
`M3_lenient`	MESA (lenient policy)	14	Withheld (**No decision**)	Removes the 6 B05/B07 defective tasks and checks B05, B07, B28a, B28b. Status: **Inadmissible** (**B28a** failed).
`ABC`	Agentic Benchmark Checklist [34]	20	Accept $\geq$ 19; Reject $\leq$ 9	Keeps all 20 source tasks; reports part scores (T: 5/8, O: 2/11, R: 1/4) and 15 unmet/unknown items as caveats without withholding the score decision.

We evaluate each case under two pinned policies (Table 2): (1) **Strict policy (`POLICY-P5-STRICT`, primary study policy):** requires level *Verified*, check outcome *passed*, scope covering the coding agent and tasks, and recorded procedure, check kind, location, and detection limits across all seven criteria. (2) **Lenient policy (`POLICY-P5-LENIENT`):** requires **B05**, **B28a**, and **B28b** at *Verified* and **B07** at *Documented* (each with outcome *passed*), and omits **B12**, **B14a**, and **B14b** to show the decision when benchmark uncertainty reporting and pretraining exposure are set aside.

3.4 Comparison Methods

Because all 20 source tasks come from SymPy, the unfiltered benchmark resolve rate (Method 1) and the repository-filtered resolve rate (Method 2, `M2`) coincide. We therefore compare **five method families producing six evaluated outputs** per coding agent on the source tasks (Table 3), each using the 95% Wilson interval of Eq. 1 at $\theta = 0.70$.

4 Prepared Evaluation

4.1 Scope and Session Budget

The empirical protocol evaluates two coding agents on SymPy at threshold $\theta = 0.70$: Gemini 3.8 Flash Medium in the Antigravity CLI (`CONFIG_01_GEMINI_FLASH_MED`) and GPT-6.1 Sol at medium reasoning effort in the Codex CLI (`CONFIG_02_SOL_MED`). Each session runs one coding agent on one task for at most 15 minutes (900 seconds) without human input. The budget allocates at most 90 sessions in a ledger: 2 pilot sessions

(1 task × 2 agents), 40 source sessions (20 tasks × 2 agents), 40 target sessions (20 tasks × 2 agents), and up to 8 reserve slots restricted to documented infrastructure interruptions (each interruption may be replaced at most once, and failed starts retain their ledger slots). No session can open until the protocol, signed forms, signed requirement, policy, and task lists are frozen into a cryptographic manifest and the author signs a phase authorization citing that manifest's SHA-256 digest.

4.2 Requirement, Source Tasks, and Target Tasks

Under the 95% Wilson interval at $\theta = 0.70$, 9 resolved tasks out of 9 yield LB = 0.701 whereas 8 out of 8 yield LB = 0.676, fixing the minimum observation count at $n_{\min} = 9$.

Source tasks ($N = 20$). The source pool comprises all 75 SymPy tasks in SWE-bench Verified (issues from 2016–2023). Eligibility testing confirmed that for all 75 tasks, every `FAIL_TO_PASS` test fails before the reference fix and all `FAIL_TO_PASS` and `PASS_TO_PASS` tests pass with the reference fix. Ordering the 75 tasks by a deterministic SHA-256 hash of a fixed label and task ID selects the first task as the pilot task and the next 20 as the source tasks (the rehearsal task `sympy__sympy-21930` is last in the ordering and outside the sample).

Target tasks ($N = 20$). The target pool comprises SymPy pull requests merged between 1 May and 5 November 2026—after the later stated training cutoff of the two models (March 2026 for Gemini 3.8 Flash; 30 April 2026 for GPT-6.1 Sol)—that close a linked issue and modify both source and test files. Of 322 merged pull requests in that window, 108 satisfied the filter; the first 20 checked in deterministic order passed the eligibility check (merged between 13 June and 5 November 2026) with zero skipped candidates. Five of those 20 pull requests close issues whose discussion threads were opened before the training cutoffs (`27443`, `13365`, `21772`, `19324`, and `26652`), so RQ2 reports target results both on all 20 target tasks and on the 15 post-cutoff-issue tasks.

4.3 Executed Verification Checks and Signed Forms

Eight MESA rating forms cover **B05** (pilot + 20 source tasks), **B07** (20 source tasks), **B12**, **B14a**, **B28a**, and **B28b** (release scope), and **B14b** (one form per coding agent configuration), alongside one ABC checklist form covering the 33 ABC items mapped to the seven criteria. Claude Sonnet 5.5 generated the eight MESA forms in two runs (A and B) from a fixed prompt and completed the ABC record in a third run. On 9 November 2026, the author verified and signed the eight MESA forms from run A, the ABC record, and the requirement record (recording 34 field-level edits between run A's drafts and the signed forms).

Executed B07 exploit check. To test whether SWE-bench Verified's hidden tests on the 20 source tasks can be passed without fixing the stated bug (**B07**), Claude Sonnet 5.5 received only the issue text and reference fix for each task, wrote an independent verification test (which had to fail pre-fix and pass post-fix), and wrote up to three candidate patches omitting part of the fix. Across the 20 source tasks, a patch passed the hidden SWE-bench regression tests while failing the independent check on **17 of 20 tasks**. Under MESA's **B07** rule, a mechanical pass counts as a task defect only when the broken behavior is stated in the issue description rather than inferred solely from

the reference fix. Both rating runs A and B and the signed **B07** form identify the same **6 tasks as task defects** (`sympy__sympy-13878`, `-15809`, `-18199`, `-21847`, `-22914`, and `-23824`) and set aside the remaining 11 tasks where the omitted behavior was not stated in the issue text. With those 6 tasks listed for removal, the signed **B07** form is rated *Verified* with check outcome *passed* on the remaining 14 tasks.

Executed B05 overbroad-test check and ABA comparison. A 2026 audit of SWE-bench Verified [21] identified `sympy__sympy-18199` as having tests wider than its issue prompt. Executing a patch restricted to the issue's stated $x \equiv 0 \pmod{p}$ bug confirmed that the hidden tests reject the valid fix. The signed **B05** form therefore rates **B05** *Verified* with outcome *passed* after listing `sympy__sympy-18199` as a task defect (noting the 2026 audit's 59.4% flaw rate on 138 tasks as a detection limit). Compared against ABA's published audit [30], which flags 2 of the 20 source tasks (`sympy__sympy-13031` and `-18199`), both methods flag `-18199`, while MESA's executed **B07** check removes 5 additional tasks that ABA did not flag.

Executed B28a grading-runner check. Because SWE-bench Verified runs the test script inside the patched repository container, Claude Sonnet 5.5 generated 9 patches across 3 source tasks representing the 3 SymPy runner layouts (`sympy__sympy-13031`, `-20590`, and `-24539`) that modified only the repository test-runner scripts without touching library code. The official SWE-bench grading harness scored all **9 of 9 patches as resolved**. Because this vulnerability is a release-wide property of the grading harness rather than a single task's test suite, the signed **B28a** form records level *Verified* with check outcome *failed*.

Remaining signed forms (`B12`, `B14a`, `B14b`, `B28b`, and `ABC`). On the remaining signed forms, **B12** is *Undocumented* with outcome *not performed* (SWE-bench reports point resolve rates without confidence intervals); **B14a** is *Verified* with outcome *failed* (published contamination audits show verbatim memorization of SWE-bench Verified items [15,21]); **B14b** for both coding agents is *Documented* with outcome *not performed* (all 2016–2023 source tasks predate both models' 2026 training cutoffs and no model-specific contamination check exists); **B28b** is *Verified* with outcome *passed* (container git history was inspected and contains no post-base-commit fix commits); and **ABC** records 8 items met, 9 not met, 10 not applicable, and 6 unknown (15 caveats).

4.4 Session Validity, Sandbox Checks, and Replay

Sandbox checks (v1–v8) ran both CLI tools in an isolated scratch workspace to test file-read and network restrictions before the rehearsal. While those scratch checks confirmed the tool flags and hook settings, runtime isolation during the actual study sessions is verified from the recorded session transcripts during the study runs. A session is marked *invalid* (retaining its slot and counting as unresolved in the primary analysis, with test-only resolution reported alongside as a control) if its transcript accesses study folders, git history beyond the base commit, network commands, external tools, post-prompt messages, or modifies test-runner scripts or test configuration files.

After source sessions complete, the *replay* tests the decision engine's implementation on the 20 collected source records of one coding agent across three conditions: (1) *Original evidence*, (2) *Provenance removed* (stripping the session log hash to verify

Table 4. Exact binomial decision probabilities and cut points under 95% Wilson bounds (θ = 0.70).

Sample Size (N)	Accept / Reject Cut Points	True p = 0.75		True p = 0.65		True p = 0.85	
		Accept	Inconcl.	Reject	Inconcl.	Accept	Inconcl.
N = 14 (`audit_only`, `M3`)	Accept 14; Reject ≤ 6	1.8%	97.2%	7.5%	92.2%	10.3%	89.7%
N = 15 (Target post-issue)	Accept ≥ 14; Reject ≤ 7	8.0%	90.3%	11.3%	87.3%	31.9%	68.1%
N = 18 (`ABA`)	Accept ≥ 17; Reject ≤ 8	3.9%	95.5%	6.0%	93.6%	22.4%	77.6%
N = 20 (`M2`, `ABC`, Target)	Accept ≥ 19; Reject ≤ 9	2.4%	97.2%	5.3%	94.5%	17.6%	82.4%
N = 30	Accept ≥ 26; Reject ≤ 16	9.8%	89.4%	12.6%	86.6%	52.4%	47.5%
N = 45	Accept ≥ 38; Reject ≤ 25	9.4%	90.2%	12.1%	87.5%	63.9%	36.1%
N = 60	Accept ≥ 49; Reject ≤ 35	14.8%	84.9%	17.1%	82.5%	81.9%	18.1%
N = 90	Accept ≥ 71; Reject ≤ 54	16.5%	83.4%	18.8%	81.1%	92.6%	7.4%
N = 120	Accept ≥ 93; Reject ≤ 74	23.3%	76.6%	25.0%	74.9%	98.1%	1.9%

that the task triggers **Inquiry Required**), and (3) *Identity mismatch* (altering task or patch linkage to verify that Level 1 rejects the case).

4.5 Planned Analysis and Known Pre-Session Outcomes

Because all nine review records and the requirement are signed prior to the coding sessions, the evidence status and task filtering of every source method are fixed in advance (Table 3):

RQ1 (Comparison across the six source outputs): Both `M3_strict` and `M3_lenient` return **Inadmissible** and withhold a score decision (**No decision**; blocking criteria in Table 3). By contrast, `M2` (N = 20), `audit_only` (N = 14, removing the 6 B05/B07 defective tasks), `ABA` (N = 18, removing 2 tasks), and `ABC` (N = 20, attaching 15 caveats) all issue a score decision (**Accept**, **Reject**, or **Inconclusive**).

RQ2 (Comparison with the post-cutoff direct test): For the four methods that issue a source decision (`M2`, `audit_only`, `ABA`, `ABC`), RQ2 compares each issued decision against the direct-test decision on the N = 20 post-cutoff target tasks (and on the N = 15 target tasks whose issues opened after the cutoffs), alongside the source-minus-target resolve-rate difference with 95% Newcombe intervals. Because `M3_strict` and `M3_lenient` return **No decision**, their withheld output is reported as *not comparable to an issued decision*, while MESA's pre-gate score decision on the 14 retained clean tasks is reported separately as a counterfactual output (`M3_withheld`, which coincides numerically with `audit_only`).

5 Completed Calculations and Simulation

5.1 Exact Binomial Decision Probabilities across Sample Sizes

Table 4 reports the exact binomial probabilities of **Accept**, **Reject**, and **Inconclusive** under the 95% Wilson interval (Eq. 1) at θ = 0.70 across the task counts of our study methods, as labeled in the table, and larger sample sizes N = 30 to 120.

At the study sample sizes N = 14, 18, and 20, a coding agent whose true resolve rate is p = 0.75 receives an **Inconclusive** decision in 97.2%, 95.5%, and 97.2% of cases, respectively, and a definitive **Accept** or **Reject** occurs only when observed performance

Table 5. False acceptance rate per case under assumed review accuracy ($N = 30, \theta = 0.70$, 10,000 trials).

Defect rate (d)	Shift (Δ)	M2 cov.	M3 cov.	M2	Random	Proximity	M3
0.10	−0.15	12.2%	6.4%	10.2%	5.4%	5.6%	4.7%
0.10	−0.05	12.9%	7.0%	10.6%	5.7%	6.3%	5.0%
0.20	−0.15	18.7%	6.7%	16.9%	6.1%	6.5%	5.2%
0.20	−0.05	17.1%	6.2%	15.5%	5.6%	6.0%	4.8%
0.30	−0.15	22.4%	6.8%	20.8%	6.4%	6.5%	5.4%
0.30	−0.05	22.2%	6.5%	20.5%	6.0%	6.3%	5.1%

lies far above or below the 0.70 threshold (for example, $\geq 19/20$ or $\leq 9/20$). Even at $p = 0.85$, reaching $> 80\%$ power for **Accept** requires $N = 60$ tasks (81.9%, with discrete binomial non-monotonicity such as 78.8% at $N = 62$).

5.2 Monte Carlo Simulation Under Assumed Review Parameters

Because the empirical SymPy case evaluates a single repository where MESA withholds its decision under both policies, we also run a synthetic Monte Carlo simulation (10,000 trials per scenario, $N = 30$, $\theta = 0.70$) to examine how the decision rules behave across varying benchmark defect rates $d \in \{0.10, 0.20, 0.30\}$ and source-to-target performance shifts $\Delta \in \{-0.15, -0.05, 0\}$. In the simulation, the baseline source resolve rate is 0.72 and rises to 0.86 when benchmark evidence is defective; the simulated review is given *assumed* parameters (85% defect detection rate, 5% false-alarm rate, and 10% missing-documentation rate). We compare MESA (`M3`) against `M2` and two selective-classification baselines that withhold the exact same fraction of decisions as `M3`: *Random* withholding and *Proximity* withholding (withholding scores closest to $\theta = 0.70$) [6]. In Table 5, *coverage* (`M2 cov.`, `M3 cov.`) denotes the proportion of simulated trials that issue a definitive **Accept** or **Reject** decision rather than **Inconclusive** or **No decision**.

At $d = 0.30$ and $\Delta = -0.15$, false acceptance drops from 20.8% under `M2` (22.4% coverage) to 5.4% under `M3` (6.8% coverage), compared with 6.4% for Random and 6.5% for Proximity withholding at the same 6.8% coverage (Table 5; mean reduction across 100 seeds is 0.88 ± 0.09 percentage points vs. Random and 1.14 ± 0.10 vs. Proximity, and across a 576-cell parameter sweep at $d = 0.30$, `M3` reduces false acceptance by 0.20 to 1.50 points over Random in all 72 noncompliant cells). Most of the reduction from 20.8% to 5.4% comes from abstaining altogether; the selective advantage over matched-coverage random abstention is approximately one percentage point, and when the target agent is compliant ($d = 0.30, \Delta = 0$), removing inflated tasks increases false rejection from 0.44% (Random) to 1.42% (`M3`).

6 Discussion: Aligning Requirements Validation with AI Velocity

6.1 Separating Automated Checks from Human Requirements Decisions

As AI coding agents accelerate patch generation, relying on unverified leaderboard resolve rates or single-pass automated task filters compresses requirements validation into a score that can hide structural measurement failures. On SWE-bench Verified (SymPy),

our executed checks show two distinct failure modes: task-level test weaknesses (**B07** cheating patches passing hidden tests on 17 of 20 tasks, with 6 breaking requirements stated in the issue text, and **B05** overbroad tests on `sympy__sympy-18199`) and release-level harness vulnerabilities (**B28a** test-runner hijacking succeeding in 9 of 9 attempts, plus **B14a** pretraining contamination).

Our three-level architecture separates what can be automated at AI speed from what requires human requirements engineering judgment. Automated tools verify file hashes, session log provenance, task-level exploit counterexamples, and Wilson confidence intervals. Conversely, human engineers specify and sign the capability threshold θ, the acceptable detection limits in the policy, and the transfer justification, and when the engine returns **Inadmissible** or **Inquiry Required** (**No decision**), it outputs the exact blocking criteria and missing checks rather than issuing an unsupported **Accept** or **Reject**.

6.2 Small-Sample Confidence Bounds in Subset Evaluations

Because running agent evaluations on full repositories is expensive, teams often evaluate subsets of $N = 14$ to 30 tasks. As shown in Table 4, when a requirement enforces a 95% Wilson confidence bound at $\theta = 0.70$, sample sizes of $N = 14$ to 20 yield $> 90\%$ **Inconclusive** outcomes for a modestly compliant agent ($p = 0.75$): an observed resolve rate of $14/20$ (70%) has a 95% lower bound of only 0.481. Reporting **Inconclusive** separately from **No decision** (withheld due to evidence defects) allows teams to see whether an undecided outcome calls for repairing the benchmark harness or running more tasks.

7 Limitations

Single-repository case, sample size, and withheld MESA decisions. The empirical protocol evaluates two coding agents on a single repository (SymPy, 20 source and 20 target tasks), and the live coding sessions and replay remain pending execution after the protocol freeze. Because the signed **B28a** form records a release-wide grading runner failure (and the strict policy is also blocked by **B14a**, **B12**, and **B14b**), both `M3_strict` and `M3_lenient` return **No decision** on the SymPy case. Consequently, the empirical case demonstrates how the engine filters defective tasks ($20 \rightarrow 14$) and explains abstention, rather than demonstrating improved decision accuracy over `M2`, `audit_only`, `ABA`, or `ABC`; for RQ2, MESA's withheld output is not comparable to an issued decision, and at $N = 14$ to 20, Wilson bounds leave most decisions near $\theta = 0.70$ **Inconclusive**.

Restricted criterion profile and source-to-target differences. Our seven-criterion profile omits MESA's construct definition criteria (**B01**–**B04**), and the engine verifies only the presence, signer, and scope match of the transfer justification rather than the empirical truth of its argument. Similarly, passing maintainer regression tests within 15 minutes is narrower than human code review, and the 2026 post-cutoff target tasks differ from the 2016–2023 SWE-bench Verified source tasks in both training exposure and issue difficulty (with 5 of the 20 target issues opened prior to the stated training cutoffs).

Model-assisted preparation, shared-scratch overlap, and single human signer. All nine review records and the requirement were verified and signed by one author. Claude Sonnet 5.5 generated the **B07** exploit patches, the **B28a** runner patches, the rating form drafts (runs A and B), and the ABC answers, while Claude Opus 5.5 (from the same model family) implemented the engine and scripts, drafted the requirement and transfer justifications, and assisted in editing this manuscript. During rating run A, the B07, B14a, B14b, B28a, and B28b sessions overlapped in a shared scratch folder, so run B was repeated cleanly after run A finished, and the ABC session read the MESA rating prompt rules before answering the ABC checklist. Finally, while sandbox checks (v1–v8) tested the CLI flags in a scratch directory, runtime network and file isolation in the study sessions remains subject to session-log verification after the sessions run.

8 Conclusion

This paper turns MESA's traceability design into an executable decision method for coding-agent requirements, separating input integrity, evidence admissibility, and Wilson score decisions while returning every blocking reason and removed task. On SWE-bench Verified (SymPy), executed verification checks identified 17 of 20 tasks where cheating patches passed the hidden tests (6 of which break issue-stated requirements and are removed as task defects alongside 1 B05 overbroad-test task) and 9 of 9 cases where test-runner edits hijacked the grading harness (**B28a**), causing both signed MESA policies to withhold a decision (**Inadmissible**) while raw resolve rates, audit-only filtering ($N = 14$), ABA ($N = 18$), and ABC ($N = 20$ with 15 caveats) issue score decisions. Executing the frozen 90-session protocol across the two coding agents on the 20 source and 20 post-cutoff target tasks is the immediate next step.

Data Availability

In accordance with the REFSQ 2027 Open Science Policy and double-blind review requirements, an anonymized replication package is provided with this submission. It contains the decision engine and schemas, the session and control scripts, the 75 source and 20 post-cutoff target SymPy task records, all 20 executed **B07** exploit attempts and 9 **B28a** runner checks, both rating runs (A and B), the nine signed forms, the signed requirement, the signing diff, the readiness report, the simulation scripts, and the automated test suite (181 unit tests). Raw session and harness logs containing local host paths are kept outside version control, with their SHA-256 digests and summaries recorded in the package.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S.: SWE-Bench+: Enhanced coding benchmark for LLMs. arXiv preprint arXiv:2410.06992 (2024), https://arxiv.org/abs/2410.06992

2. Arief, M., Khatim, N.A., Akarma, A., Irfan, A.A.A.: Integrating high-level requirements to low-level tests with machine-readable V&V specifications. arXiv preprint arXiv:2607.17686 (2026)
3. Badertdinov, I., Golubev, A., Nekrashevich, M., et al.: SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv preprint arXiv:2505.20411 (2025)
4. Brown, L.D., Cai, T.T., DasGupta, A.: Interval estimation for a binomial proportion. Statistical Science **16**(2), 101–133 (2001). https://doi.org/10.1214/ss/1009213286
5. Ding, F.: What the specification did not say: Specification-oracle gaps in test-graded coding-agent benchmarks. arXiv preprint arXiv:2610.30635 (2026)
6. El-Yaniv, R., Wiener, Y.: On the foundations of noise-free selective classification. Journal of Machine Learning Research **11**, 1605–1641 (2010)
7. Ghosh, A., Reuel, A., Chim, J., et al.: Evaluation cards: An interpretive layer for AI evaluation reporting. arXiv preprint arXiv:2606.09809 (2026)
8. Goswami, A.: Assurance envelopes for autonomous coding agents: Minimum-cost evidence for software change. arXiv preprint arXiv:2609.16302 (2026)
9. e Habiba, U., Haug, M., Bogner, J., Wagner, S.: How mature is requirements engineering for ai-based systems? a systematic mapping study on practices, challenges, and future research directions. Requirements Engineering **29**, 567–600 (2024). https://doi.org/10.1007/s00766-024-00432-3
10. Hawkins, R., Paterson, C., Picardi, C., Jia, Y., Calinescu, R., Habli, I., McDermid, J.: Guidance on the assurance of machine learning in autonomous systems (AMLAS). arXiv preprint arXiv:2102.01564 (2021)
11. Hou, X., Zhao, Y., Liu, Y., et al.: Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology (2024). https://doi.org/10.1145/3689498
12. Kane, M.T.: Validating the interpretations and uses of test scores. Journal of Educational Measurement (2013). https://doi.org/10.1111/jedm.12000
13. Kokaly, S., Salay, R., Chechik, M., Lawford, M., Maibaum, P.E.: Safety case impact assessment in automotive software systems: An improved model based approach. In: International Conference on Computer Safety, Reliability, and Security (SAFECOMP). pp. 69–85. Springer (2017). https://doi.org/10.1007/978-3-319-66266-4_5
14. Lewis, G.A., Brower-Sinning, R., Edman, R., Ozkaya, I., Echeverría, S., Derr, A., Beaudoin, C., Maffey, K.R.: Quality model for machine learning components. arXiv preprint arXiv:2602.05043 (2026)
15. Liang, S., Garg, S., Moghaddam, R.Z.: The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. arXiv preprint arXiv:2506.12286 (2025), https://arxiv.org/abs/2506.12286
16. Liu, Y.L., Blodgett, S.L., Cheung, J.C.K., Liao, Q.V., Olteanu, A., Xiao, Z.: ECBD: Evidence-centered benchmark design for NLP. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 16349–16365. Association for Computational Linguistics, Bangkok, Thailand (Aug 2024). https://doi.org/10.18653/v1/2024.acl-long.861
17. Mohl, J., Gardner-Challis, N., Dubois, M., et al.: Automated transcript analysis for detecting flaws in agentic benchmarks. arXiv preprint arXiv:2607.27518 (2026). https://doi.org/10.48550/arXiv.2607.27518
18. Mukhopadhyay, A., Nalla, S.R.: Reusing assurance evidence in AI pipelines: Validity conditions, a fail-closed release gate, and a replay of 714 public data version control pipelines. arXiv preprint arXiv:2610.27672 (2026)

19. National Institute of Standards and Technology: Practices for automated benchmark evaluations of language models. NIST Special Publication (Initial Public Draft) AI 800-2 ipd, National Institute of Standards and Technology (2026). https://doi.org/10.6028/NIST.AI.800-2.ipd
20. OpenAI: Separating signal from noise in coding evaluations: An audit of SWE-bench pro. https://openai.com/index/separating-signal-from-noise-coding-evaluations/ (Jul 2026)
21. OpenAI: Why SWE-bench verified no longer measures frontier coding capabilities. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/ (Feb 2026)
22. Picardi, C., Hawkins, R., Paterson, C., Habli, I.: Transfer assurance for machine learning in autonomous systems. In: Proceedings of the Workshop on Artificial Intelligence Safety (SafeAI) at AAAI. CEUR Workshop Proceedings, vol. 3381 (2023)
23. Pizarro, E., Azanza, M., Lamancha, B.P.: Beyond the hype: Enabling informed LLM adoption in industry through systematic evaluation. Science of Computer Programming **254**, 103530 (2026). https://doi.org/10.1016/j.scico.2026.103530
24. Reuel, A., Hardy, A., Smith, C., Lamparth, M., Hardy, M., Kochenderfer, M.J.: Betterbench: Assessing ai benchmarks, uncovering issues, and establishing best practices. In: Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS) (2024), https://arxiv.org/pdf/2411.12990
25. Singh, S., Nan, Y., Wang, A., D'souza, D., Kapoor, S., et al.: The leaderboard illusion. arXiv preprint arXiv:2504.20879 (May 2025), https://arxiv.org/pdf/2504.20879
26. Singh, V., Kaewprapha, P.: From model proliferation to deployment intelligence: A risk-traceable system-aware model selection framework. Applied Sciences **16**(21), 10447 (2026). https://doi.org/10.3390/app162110447
27. Šljivo, I., Gallina, B., Carlson, J., Hansson, H.: Generation of safety case argument-fragments from safety contracts. In: International Conference on Computer Safety, Reliability, and Security (SAFECOMP). Lecture Notes in Computer Science, vol. 8666, pp. 170–185. Springer (2014). https://doi.org/10.1007/978-3-319-10506-2_12
28. Tu, X., Wang, T., Lu, Y., Huang, K., Qu, Y., Mostafavi, S.: BenchGuard: Who guards the benchmarks? automated auditing of LLM agent benchmarks. arXiv preprint arXiv:2604.24955 (2026)
29. Wang, H., Li, H., Mang, Q., Cheung, A., Sen, K., Song, D.: Do androids dream of breaking the game? systematically auditing AI agent benchmarks with BenchJack. arXiv preprint arXiv:2605.12673 (2026), https://arxiv.org/abs/2605.12673
30. Wang, J., Bianchi, F., Zhu, S., Nie, F., Kwon, Y., Dhingra, B., Zou, J.: Automated benchmark auditing for AI agents and large language models. arXiv preprint arXiv:2605.26079 (2026)
31. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association **22**(158), 209–212 (1927). https://doi.org/10.1080/01621459.1927.10502953
32. Yu, B., Zhu, Y., He, P., Kang, D.: UTBoost: Rigorous evaluation of coding agents on SWE-Bench. In: Annual Meeting of the Association for Computational Linguistics (ACL 2025) (2025), https://arxiv.org/abs/2506.09289
33. Zhang, L., He, S., Zhang, C., et al.: SWE-bench goes live! arXiv preprint arXiv:2505.23419 (2025)
34. Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., et al.: Establishing best practices in building rigorous agentic benchmarks. In: Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track (2025)